

MODUL III

CSS (CASCADING STYLE SHEETS)

3.1 Tujuan Praktikum

Tujuan dari praktikum ini adalah sebagai berikut:

1. Mahasiswa dapat memahami konsep dasar CSS dalam pengembangan web.
2. Mahasiswa dapat memahami cara menghubungkan CSS dengan HTML.
3. Mahasiswa dapat menerapkan *styling* dasar pada halaman web.

3.2 Dasar Teori

Cascading Style Sheets (CSS) adalah bahasa yang digunakan untuk mendeskripsikan tampilan suatu dokumen yang ditulis dalam HTML atau XML, termasuk pengaturan tata letak, warna, dan jenis huruf (W3Schools, 2026). Berbeda dari HTML yang berfungsi menyusun struktur dan konten halaman, CSS berperan murni pada aspek visual, sehingga satu dokumen HTML dapat ditampilkan dengan berbagai gaya berbeda hanya dengan mengganti berkas CSS yang diterapkan padanya (Mozilla, 2026). Kehadiran CSS memungkinkan pengembang memisahkan urusan konten dan tampilan, sebuah prinsip yang dikenal sebagai *separation of concerns*, sehingga proses pengembangan dan pemeliharaan situs web menjadi lebih terstruktur.

CSS berfungsi memisahkan struktur dan desain sebuah halaman web. Berkas HTML bertugas membawa konten sekaligus menentukan struktur dokumen, sedangkan berkas CSS murni mengatur cara konten tersebut ditampilkan (Mozilla, 2026). Pemisahan ini membuat kode lebih mudah dirawat karena perubahan tampilan cukup dilakukan pada satu tempat tanpa menyentuh kerangka HTML. Dengan *style sheet* eksternal, tampilan seluruh situs web bahkan dapat diubah hanya dengan menyunting satu berkas saja (W3Schools, 2026). Struktur yang ringkas juga membuat halaman lebih cepat dimuat dan memudahkan pembagian kerja ketika pengembangan dilakukan secara tim.

Terdapat tiga cara untuk menerapkan CSS pada dokumen HTML. Pertama, inline CSS, yaitu menuliskan aturan gaya langsung pada atribut style di dalam tag HTML tertentu, cocok digunakan untuk pengujian cepat atau pengaturan yang hanya berlaku pada satu elemen saja (W3Schools, 2026). Kedua, internal CSS, yaitu menempatkan seluruh aturan gaya di dalam elemen `<style>` pada bagian `<head>` dokumen, sehingga gaya tersebut hanya berlaku untuk halaman yang bersangkutan. Ketiga, external CSS, yaitu menuliskan aturan gaya pada berkas terpisah berekstensi .css yang kemudian dihubungkan ke dokumen HTML melalui elemen `<link>`, dan cara ini paling direkomendasikan karena satu berkas CSS dapat digunakan kembali oleh banyak halaman sekaligus (Mozilla, 2026). Dari ketiga pendekatan tersebut, external CSS memberikan tingkat efisiensi tertinggi dalam pengelolaan gaya pada proyek berskala besar.

Aturan CSS terdiri atas dua komponen utama, yaitu selector dan blok deklarasi. Selector menentukan elemen HTML mana yang akan dikenai gaya, sedangkan blok deklarasi berisi satu atau lebih pasangan property dan value yang diapit tanda kurung kurawal, dengan setiap pasangan diakhiri tanda titik koma (W3Schools, 2026). Sebagai contoh, aturan `p { color: blue; font-size: 16px; }` akan menerapkan warna biru dan ukuran huruf 16 piksel pada seluruh elemen paragraf. Selector sendiri memiliki beberapa jenis, mulai dari selector elemen, class, id, hingga selector gabungan yang memungkinkan pengembang menargetkan elemen secara lebih spesifik sesuai kebutuhan desain (W3Schools, 2026)

3.3 Hasil dan Pembahasan

Praktikum pada MODUL III merujuk pada modul praktikum DPW dengan materi CSS. Kasus yang diangkat tetap sama dengan praktikum sebelumnya, yaitu halaman perpustakaan, sehingga fokus pengerjaan kali ini adalah memberikan gaya tampilan pada kerangka HTML yang sudah ada. Editor yang digunakan adalah Visual Studio Code. Hasil tampilan dilihat melalui *browser* Google Chrome. Berikut adalah penjelasan pelaksanaan praktikum beserta hasilnya.

3.3.1 Cara Menghubungkan CSS ke HTML

Berikut adalah cara menghubungkan CSS ke HTML dengan cara *inline* CSS, *internal* CSS, dan *external* CSS.

1. Cara pertama adalah *inline* CSS, yaitu menuliskan aturan gaya langsung pada atribut `style` di tag pembuka elemen. Cara ini paling cepat diterapkan karena cukup menambahkan atribut pada elemen yang dituju tanpa membuat berkas baru. Sebagai contoh, membuat berkas `inline.html` yang berisi dua paragraf, satu paragraf dengan atribut `style` dan satu lagi tanpa atribut sebagai pembanding. Kode lengkapnya dituliskan seperti berikut.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="utf-8" />
  <title>Inline CSS</title>
</head>
<body>
  <p style="color: #0f172a; font-weight: bold;"> CUMA CONTOH.
</p>
  <p>GATAU MAU NGETIK APA .</p>
</body>
</html>
```

2. Deklarasi `color: #2563eb;` pada atribut `style` memberi warna biru pada teks paragraf pertama, sedangkan `font-weight: bold;` membuat teksnya tebal. Paragraf kedua tidak membawa atribut

sehingga tampil dengan gaya bawaan *browser*.

CUMA CONTOH.

GATAU MAU NGETIK APA .

Gambar 3.1 Tampilan paragraf dengan *inline* CSS.

3. Cara kedua adalah *internal* CSS yang dituliskan di dalam elemen `<style>` pada bagian `<head>` dokumen. Aturan pada cara ini berlaku untuk seluruh elemen di satu halaman, sehingga cocok digunakan ketika sebuah halaman membutuhkan gaya khusus yang berbeda dari halaman lain. Penulis menuliskan berkas `internal.html` dengan aturan warna untuk elemen `<h1>` dan `<p>` seperti pada kode berikut.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="utf-8" />
  <title>caSCADING STYLEE</title>
  <style>
    h1 { color: #0f172a; }
    p { color: #475569; }
  </style>
</head>
<body>
  <h1>CASCADING STYLEEE</h1>
  <p>Gaya Penulisan sajaa.</p>
</body>
</html>
```

4. Judul tampil berwarna biru gelap dan paragraf berwarna abu-abu sesuai aturan di dalam elemen `<style>`.

CASCADING STYLEEE

Gaya Penulisan sajaa.

Gambar 3.2 Tampilan halaman dengan *internal* CSS.

5. Cara ketiga adalah *external* CSS, cara yang paling sering digunakan karena aturan gaya disimpan pada berkas terpisah dan dapat dipakai bersama oleh banyak halaman.
6. buat *file* baru bernama `style.css` di folder yang sama dengan *file* HTML. Di dalamnya, tulis aturan CSS sederhana sebagai berikut.

```
body {
```

```

background-color: #f0f8ff;
font-family: Arial, sans-serif;
}

h1 {
  color: steelblue;
  text-align: center;
}

p {
  color: #333;
  font-size: 16px;
}

```

7. buat *file* `index.html` yang berisi struktur dasar HTML, lalu di dalam `<head>` tambahkan tag `<link rel="stylesheet" href="style.css">` untuk menghubungkan HTML dengan *file* CSS tersebut.

8. Dalam `<body>`, tambahkan elemen seperti `<h1>Halo Dunia!</h1>` dan `<p>Ini contoh external CSS.</p>`. Simpan kedua *file*, lalu buka `index.html` di browser. Berikut adalah kode lengkapnya.

```

<!DOCTYPE html>
<html>
<head>
  <title>aku suka DPW</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>tapi boong</h1>
  <p>tugas DPW.</p>
</body>
</html>

```

9. Berikut adalah hasil *render* dari *browser*.



Gambar 3.3 Hasil *render external* CSS.

3.3.2 Struktur Dasar CSS

Setiap aturan CSS terdiri dari dua bagian pokok, yaitu selector dan blok deklarasi. Selector berfungsi menentukan elemen HTML mana yang akan diberi gaya, sementara blok deklarasi yang diapit tanda kurung kurawal {...} memuat pasangan property dan value yang mengatur gaya tersebut. Penulisan aturannya dituliskan seperti berikut.

```
h1 {  
  color: #0f172a;  
  font-size: 24px;  
}
```

1. Pada aturan tersebut, `h1` berperan sebagai *selector*, `color` merupakan *property*, dan `#0f172a` adalah *value*-nya. Tanda titik koma wajib dituliskan di akhir setiap deklarasi agar aturan berikutnya dapat dibaca dengan benar. Beberapa jenis *selector* dasar yang dipakai pada praktikum ini dirangkum sebagai berikut.
2. *Selector universal* dituliskan dengan tanda (*) dan memilih seluruh elemen pada halaman.
3. *Selector element* dituliskan dengan nama tag seperti `p` dan `h1` serta memilih elemen berdasarkan nama tag.
4. *Selector class* dituliskan dengan tanda titik seperti `.card` dan dapat dipakai berulang pada banyak elemen.
5. *Selector ID* dituliskan dengan tanda pagar seperti `#navbar` dan bersifat unik karena hanya satu per halaman.
6. Jenis *class* menjadi pilihan utama untuk memberi gaya pada komponen karena dapat dipakai berulang, sedangkan ID dipakai untuk elemen yang unik seperti judul utama halaman.

3.3.3 Implementasi Styling Dasar

Pengaturan tampilan paling mendasar dalam CSS mencakup warna, huruf, dan ruang. Properti `color` digunakan untuk mengatur warna teks, sementara `background-color` mengatur warna latar suatu elemen. Kedua properti ini bisa diisi dengan nilai berupa kode HEX seperti `#0f172a`, fungsi `rgb()`, ataupun nama warna langsung. Untuk pengaturan huruf, `font-family` menentukan jenis huruf yang dipakai, `font-size` mengatur ukurannya,

font-weight mengatur ketebalannya, dan line-height mengatur jarak antarbaris agar teks lebih nyaman dibaca. Berikut adalah contoh bagaimana cara mengimplementasikan *styling* dasar CSS.

1. Ruang di sekeliling elemen diatur melalui *box model*. Properti Margin berfungsi menciptakan jarak di bagian luar border sehingga suatu elemen tidak menempel langsung dengan elemen lain di sekelilingnya. Padding berperan sebaliknya, yaitu menyediakan ruang kosong di antara batas border dengan konten yang ada di dalamnya, seperti teks atau gambar. Sementara itu, border membentuk garis pembatas kotak elemen yang dapat diatur ketebalan, gaya garis, maupun warnanya sesuai kebutuhan desain (Mozilla, 2026).

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="utf-8" />
<title>Inline CSS</title>
</head>
<body>
<p style=" color: #0f172a; font-weight: bold;">
CUMA CONTOH.
</p>
<p>GATAU MAU NGETIK APA .</p>
</body>
</html>

<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="utf-8" />
<title>caSCADING STYLEE</title>
<style>
h1 { color: #0f172a; } p { color: #475569; }
</style>
</head>
<body>
<h1>CASCADING STYLEEE</h1>
<p>Gaya Penulisan sajaa.</p>
</body>
</html>
```

2. Halaman demonstrasi memakai internal CSS karena aturannya memang hanya untuk satu halaman. Judul tampil navy gelap, paragraf berwarna abu-abu netral dengan jarak baris 1.6, dan satu kotak

biru menampilkan kombinasi latar, padding, border, serta margin.



Gambar 3.4 Tampilan halaman demonstrasi *styling* dasar.

3.3.4 Latihan Praktikum

Latihan praktikum mengikuti urutan materi pada modul, yaitu penyusunan kartu produk. Latihan dituliskan lengkap mulai dari `<!DOCTYPE html>` beserta tangkapan layar hasilnya. Setiap langkah pengerjaan juga diuraikan terlebih dahulu sebelum kode lengkapnya dituliskan.

1. Meminta pembangunan komponen kartu produk modern. Ketentuan pada modul meliputi penggunaan `box-sizing: border-box`, sudut membulat `border-radius: 12px`; dengan bayangan `box-shadow` yang halus, pemanfaatan *flexbox* untuk merapikan *badge* harga dan tombol aksi, serta efek `:hover` dengan `transition: 0.3s`; pada tombol. Sesuai instruksi tambahan, kartu dibuat sebanyak 3 buah. Langkah pengerjaannya adalah sebagai berikut.
2. Pasang *CSS reset* `box-sizing: border-box` beserta `margin: 0` dan `padding: 0` pada *universal selector*.
3. Susunlah tiga kartu produk berisi foto, judul, deskripsi, *badge* harga, dan tombol aksi.
4. Aturlah `border-radius`, `box-shadow`, serta perataan *flexbox* pada baris aksi.
5. Tambahkan efek `:hover` dan `transition: 0.3s` pada tombol, dimulai dari HTML.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="utf-8" />
  <title>Kartu Produk - Laundry</title>
  <link rel="stylesheet" href="style2.css">
</head>
<body>
  <main class="daftar-produk">
```



```

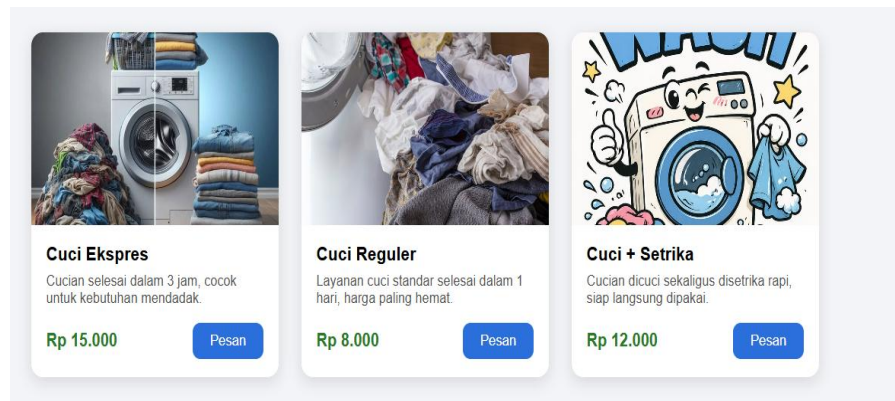
<article class="kartu-produk">
    
    <div class="konten">
        <h2>Cuci Ekspres</h2>
        <p>Cucian selesai dalam 3 jam, cocok
untuk kebutuhan mendadak.</p>
        <div class="aksi">
            <span class="harga">Rp 15.000</span>
            <button type="button" class="btn-
pesan">Pesan</button>
        </div>
    </div>
</article>
<article class="kartu-produk">
    
    <div class="konten">
        <h2>Cuci Reguler</h2>
        <p>Layanan cuci standar selesai dalam 1
hari, harga paling hemat.</p>
        <div class="aksi">
            <span class="harga">Rp 8.000</span>
            <button type="button" class="btn-
pesan">Pesan</button>
        </div>
    </div>
</article>
<article class="kartu-produk">
    
    <div class="konten">
        <h2>Cuci + Setrika</h2>
        <p>Cucian dicuci sekaligus disetrika
rapi, siap langsung dipakai.</p>
        <div class="aksi">
            <span class="harga">Rp 12.000</span>
            <button type="button" class="btn-
pesan">Pesan</button>
        </div>
    </div>
</article>
</main>
</body>
</html>

```

6. Kemudian, CSS.

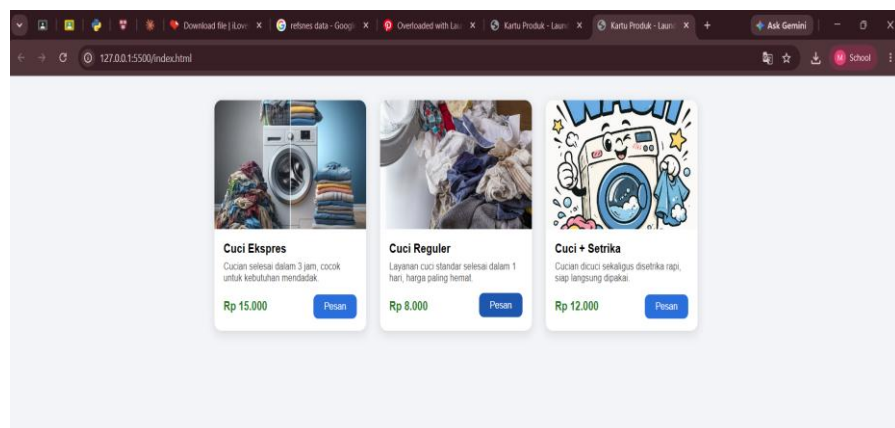
```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
body {
  font-family: Arial, Helvetica, sans-serif;
  background: #f2f4f7;
  padding: 32px;
}
.daftar-produk {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
  justify-content: center;
}
.kartu-produk {
  background: #fff;
  width: 260px;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 4px 14px rgba(0, 0, 0, 0.12);
}
.foto-produk {
  width: 100%;
  height: 170px;
  object-fit: cover;
  display: block;
}
.konten {
  padding: 16px;
}
.konten h2 {
  font-size: 17px;
  margin-bottom: 6px;
}
.konten p {
  font-size: 13px;
  color: #555;
  margin-bottom: 14px;
}
.aksi {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

7. Tiga kartu ditata berdampingan di tengah menggunakan `flex`.



Gambar 3.5 Tampilan tiga kartu produk pada latihan praktikum.

8. Tampilan saat *hover* dan *transition*. Bekerja saat kursor diarahkan ke salah satu *button* dengan efek memudarkan warna pada *button* perlahan dari biru terang menjadi gelap dalam waktu 0.3 detik.



Gambar 3.6 Kondisi *button* yang berubah warna saat kursor diarahkan.

3.3.5 Penjelasan

Berdasarkan hasil praktikum yang sudah dijalankan, halaman kartu produk berhasil menampilkan tiga kartu sejajar di tengah halaman, yaitu "Cuci Ekspres", "Cuci Reguler", dan "Cuci + Setrika". Ketiganya memiliki struktur yang seragam, terdiri dari gambar, judul, deskripsi, badge harga, dan tombol "Pesan". Susunan ini dibangun menggunakan elemen `<main class="daftar-produk">` sebagai wadah utama, dengan tiga `<article class="kartu-produk">` sebagai masing-masing item kartu.

Dari segi CSS, tampilan kartu diatur lewat beberapa properti kunci. Selektor universal (*) diberi box-sizing: border-box, margin: 0, dan padding: 0 agar perhitungan ukuran setiap elemen lebih konsisten dan jarak bawaan browser tidak lagi mengganggu tata letak. Elemen body menggunakan font Arial dengan warna latar #f0f4f8. Wadah .daftar-produk diatur dengan display: flex, justify-content: center, gap: 24px, dan padding: 40px 24px, sehingga kartu-kartu tersusun berjejer secara horizontal dengan jarak antar kartu yang rapi.

Setiap .kartu-produk memiliki lebar 260px, latar berwarna putih, sudut membulat (border-radius: 12px), padding 16px, serta bayangan (box-shadow) yang lembut. Gambar di dalam kartu diberi width: 100% dan border-radius: 8px, sementara ukuran dan warna teks judul maupun paragraf disesuaikan agar nyaman dibaca.

Pada bagian bawah kartu, .aksi menggunakan flexbox dengan justify-content: space-between sehingga *badge* harga berada di kiri dan tombol “Pesan” di kanan. *Badge* harga diberi latar #e2e8f0, teks #0f172a, dan border-radius: 999px. Tombol “Pesan” diberi latar #2563eb, teks putih, tanpa border, border-radius: 8px, dan cursor: pointer. Efek interaktif ditambahkan melalui .aksi button:hover yang mengubah latar menjadi #1e40af, serta transition: 0.3s agar perubahan warna berjalan halus. Hasilnya, saat kursor diarahkan ke tombol, warna tombol berubah secara perlahan dari biru terang ke biru gelap, seperti terlihat pada Gambar 3.6. Secara keseluruhan, latihan ini menunjukkan bahwa CSS mampu mengubah kerangka HTML sederhana menjadi tampilan kartu produk yang rapi, modern, dan interaktif.

3.3.6 Kesimpulan

Praktikum ini memperlihatkan bahwa CSS memiliki peran besar dalam meningkatkan tampilan, kenyamanan pengguna, sekaligus kesan profesional pada sebuah sistem web. Dari sisi tampilan, halaman perpustakaan yang awalnya polos berubah menjadi lebih berwarna dan tertata rapi hanya dengan aturan-aturan sederhana seperti `background-color`, `font-family`, dan `border`. `Box model` memberikan kontrol jarak yang presisi sehingga setiap komponen memiliki ruangnya masing-masing, sementara `flexbox` memungkinkan kartu produk tersusun sejajar secara otomatis tanpa perlu aturan CSS tambahan.

Dari sisi kenyamanan pengguna, penggunaan huruf Arial dengan jarak baris (`line-height`) 1.6 membuat teks lebih nyaman dibaca, efek `:hover` pada tombol memberikan kesan responsif saat berinteraksi, dan posisi kartu yang sejajar memudahkan pengunjung membandingkan produk satu sama lain. Konsistensi tampilan juga tetap terjaga karena satu berkas `style.css` yang sama dipakai di banyak halaman, sehingga setiap perubahan gaya langsung berlaku di seluruh situs tanpa perlu diubah satu per satu.

Tampilan yang konsisten dan rapi ini pada akhirnya mencerminkan profesionalitas sebuah web. Kartu produk yang menarik dan tombol yang responsif membuat situs terlihat lebih layak dipercaya, sementara pemisahan antara struktur (HTML) dan tampilan (CSS) menunjukkan bahwa kode yang dibangun tertata dan mudah dirawat.